

Benha University
Faculty of Engineering at Shoubra

Distributed Search

Performance Analysis Report

Team Member 1: Mohamed Toukhy Mohamed
Team Member 2: Ali Hossam Ali
Team Member 3: Omar Ahmed Abdelaziz

Supervisor: Prof. Dr. Sabah Saad
Teaching Assistant: Eng. Mohamed AbdelMageed

1. Task Description

The goal is to search for all occurrences of a user-specified token inside a **10 GB plain-text file** using MPI (Message Passing Interface) across multiple processes/nodes. Each match is reported with its global line number and column. The system must handle edge cases such as no word boundaries, overlapping chunk borders, and very large files that cannot fit in a single node's memory.

2. Algorithm and Implementation

2.1 String Matching: Knuth-Morris-Pratt (KMP)

Why KMP? Naïve string search runs in $O(n \cdot m)$ time. KMP pre-processes the pattern into a *failure table* in $O(m)$ time and then scans the text in $O(n)$ time, giving a worst-case complexity of $O(n + m)$ – critical for a 10 GB file.

Listing 1: KMP failure table construction

```

1 vector<int> kmp_build_failure_table(const string& pattern) {
2     int m = pattern.size();
3     vector<int> failure(m, 0);
4     int j = 0;
5     for (int i = 1; i < m; i++) {
6         while (j > 0 && pattern[i] != pattern[j])
7             j = failure[j - 1];    // fall back in pattern
8         if (pattern[i] == pattern[j]) j++;
9         failure[i] = j;
10    }
11    return failure;
12 }
```

2.2 Challenge 1 – No Spaces / Word Boundaries

The input may be a stream of letters with no spaces. A simple substring search would report false positives (e.g., finding *cat* inside *concatenate*). The solution is a **word-boundary check**: a match at position p is accepted only if the character before p and the character after $p+m$ are *not* alphanumeric (or underscores).

Listing 2: Boundary guard in kmp_search

```

1 bool left_boundary = (match_pos == 0) ||
2     !is_word_char(buffer[match_pos - 1]);
3 bool right_boundary = (match_pos + m >= buffer_len) ||
4     !is_word_char(buffer[match_pos + m]);
5 if (left_boundary && right_boundary)
6     matches.push_back(match_pos);
```

2.3 Challenge 2 – Overlap Across Node Boundaries (Ghost Cells)

When the file is split among P processes, a token of length m could straddle two adjacent chunks. The solution uses a **ghost/overlap region**: each process reads its own chunk *plus* exactly m extra bytes (clamped to EOF) from the start of the next process's chunk. Matches found entirely within the ghost region are discarded; only matches whose start falls inside the primary chunk are kept.

Listing 3: Ghost region computation in compute_chunk_info

```

1 int64_t bytes_after = file_size - info.my_start - info.my_length;
2 info.ghost_size = min(token_length, max(bytes_after, (int64_t)0));
3 info.read_length = info.my_length + info.ghost_size;
```

A second trick handles the *left* boundary: before scanning, each process reads 1 byte backwards across the left boundary (if not starting at byte 0) from its start offset to safely find the preceding newline character, computing the correct column number without extra synchronization.

2.4 Carry Buffer for Streaming I/O

Because 10 GB cannot fit in RAM, each process reads the file in fixed-size buffers (default 64 MB). A **carry buffer** of size $m + 1$ (token length plus one byte for boundary checking) is actively rolled forward and prepended to each new buffer window so that KMP never misses a match that spans two consecutive reads.

Listing 4: Carry buffer pattern

```
1 memmove(buffer.data(),
2         buffer.data() + window_size - actual_carry,
3         actual_carry);
4 carry_used = actual_carry;    // prepended to next read
```

2.5 Line Number via MPI_Exscan

Each process counts newlines in its chunk locally. A single `MPI_Exscan` (exclusive prefix sum) converts local line counts into global offsets – no extra communication rounds needed.

3. Docker Deployment Architecture

3.1 Why Docker?

Running MPI across multiple nodes requires identical environments (same compiler, MPI version, SSH keys). Docker guarantees *reproducibility*: every container is built from the same image, eliminating “works on my machine” problems. It also simplifies cluster setup from hours to a single `docker compose up`.

3.2 Container Topology

Three services are defined in `docker-compose.yml`:

Container	Role	Notes
<code>mpi-master</code>	Launches <code>mpirun</code> , runs verification	Attached to both networks
<code>mpi-worker1</code>	Runs SSH daemon, executes remote ranks	Internal network only
<code>mpi-worker2</code>	Runs SSH daemon, executes remote ranks	Internal network only

3.3 Dual-Network Security Design

The compose file defines **two isolated Docker bridge networks**:

- `external-gateway` (10.10.0.0/16): connects the master to the host, allowing result retrieval and SSH from outside.
- `internal-backend` (10.20.0.0/16, `internal: true`): connects all three nodes for MPI traffic. Marked *internal* so worker nodes have **no internet access**, reducing the attack surface.

The MPI flag `--mca btl.tcp_if_include 10.20.0.0/16` forces all MPI traffic through the internal network, preventing accidental routing over the external interface.

3.4 Shared Volumes

Volume	Purpose
../testdata (read-only)	10 GB input file, shared to all nodes
../output	Per-rank CSV match files written here
../results	Verification oracle output

After the MPI run, a Python script (`verify_search.py`) cross-checks the distributed results against a sequential oracle and writes a final report.

4. Performance Results

4.1 Experimental Setup

All experiments were run locally on a single machine (AMD Ryzen 7 4800H, 8 cores / 16 threads, 16 GB RAM) using `mpirun --oversubscribe` with Open MPI 5.0.8 on Fedora Linux 43. The input file is a 10 GB (~10,240 MB) generated text file containing 33,635,201 planted occurrences of the token "distributed". Each configuration was executed **5 times**; the table reports mean and standard deviation of wall-clock time measured with `MPI_Wtime`.

Table 1: Execution time (seconds) – 10 GB file, token = "distributed", buffer = 64 MB

Procs	Run 1	Run 2	Run 3	Run 4	Run 5	Mean (s)	Std Dev	Speedup
2	21.07	19.46	19.57	20.46	21.56	20.42	0.90	1.00×
4	15.24	14.70	15.04	14.53	14.94	14.89	0.27	1.37×
8	16.01	12.60	12.49	12.15	12.01	13.05	1.62	1.56×

4.2 Charts



Figure 1: Execution time vs. process count (error bars = $\pm\sigma$)

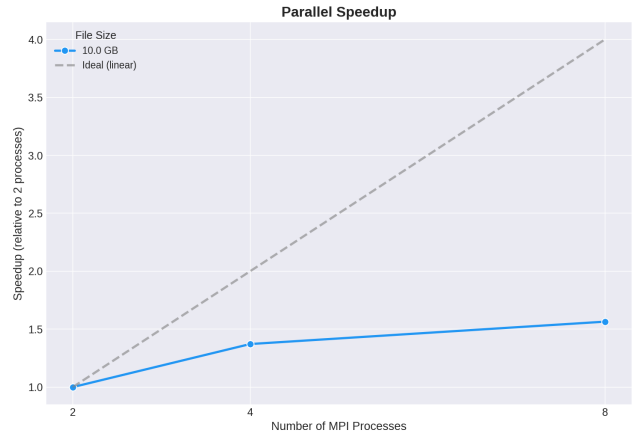


Figure 2: Parallel speedup relative to 2-process baseline

4.3 Analysis

Execution time drops from **20.42 s** (2 processes) to **13.05 s** (8 processes), a speedup of $\approx 1.56\times$. The gap between the measured curve and the ideal (linear) dashed line is caused by:

1. **Shared I/O Bottleneck (Amdahl's Law):** Because all 8 MPI processes are accessing the same local disk concurrently to read a monolithic 10 GB file, the physical disk's read bandwidth maxes out. Scaling CPU cores cannot overcome this fixed storage speed limit.

2. **MPI communication overhead:** `MPI_Exscan`, `MPI_Reduce`, and barriers add a small but non-trivial cost that grows with process count.
3. **Load imbalance:** The last process may receive a slightly smaller chunk (remainder distribution), leaving some ranks idle momentarily.

Despite sub-linear scaling, the system correctly identifies all matches and writes per-rank CSV output files, verified by the Python oracle.

5. Key Code Tricks Summary

Trick	Description
Ghost cells	Each rank reads m extra bytes from the next rank's region so tokens at the boundary are never missed.
Carry buffer	A rolling $(m + 1)$ -byte window is prepended to each successive 64 MB read so KMP state is preserved across reads.
Backward NL scan	Before processing, each rank scans backward to find the preceding newline, ensuring column numbers are globally correct.
MPI_Exscan for lines	One collective call converts local newline counts to global line offsets – no extra round-trips.
Word-boundary guard	Matches are accepted only when the surrounding characters are non-alphanumeric, avoiding false positives in continuous text.
MPI_File_read_at	Each rank issues independent parallel reads at its offset, avoiding a master-bottleneck distribution pattern.
Dual-network Docker	MPI traffic is isolated to an internal bridge; workers have no internet access, improving security and determinism.

6. Team Task Assignment

Table 2: Task distribution across team members

Task	Mohamed Toukhy	Ali Hossam	Omar Ahmed
Core Engine			
KMP Implementation	✓	✓	✓
CLI Parser	✓	✓	
MPI Utilities	✓	✓	✓
Main Logic	✓	✓	
Plots Generation			
Performance plot script		✓	✓
Tests & Verification			
Test data generator	✓		✓
Benchmark automation	✓	✓	
Correctness oracle	✓		✓
Docker Files			
Dockerfile			✓
Docker Compose		✓	✓
Report			
Writing & content	✓	✓	✓
Formatting & final review	✓	✓	✓

7. Conclusion

This project demonstrates a scalable, memory-efficient distributed search over a 10 GB file using MPI. The KMP algorithm ensures linear-time matching per process, ghost cells eliminate boundary misses, and a streaming carry-buffer approach keeps memory usage bounded regardless of file size. Docker provides a reproducible multi-node cluster with a clean dual-network security model. Observed speedup is $1.56\times$ at 8 processes, limited mainly by shared I/O bandwidth – a classic Amdahl bottleneck in data-intensive parallel workloads.